COASTEL SEVEN

Career Portal

Product Requirements Document & Technical Guide

Version 2.0 | April 2026

Includes: API Reference | Backend Flow | DB Schema | Junior Dev Guide

Prepared for: Coastel Seven | Confidential — Internal Use Only

Document Information

Document Title	Career Portal — PRD & Technical Reference v2.0
Version	2.0 (added API Reference, Backend Flow, DB Schema, Dev Guide)
Status	Draft — Awaiting Stakeholder Approval
Client / Company	Coastel Seven
Prepared By	Project Team
Date	April 2026
Classification	Confidential — Internal Use Only

1. Executive Summary

Coastel Seven requires a modern, AI-powered Career Portal to streamline its hiring process. The portal serves two distinct candidate segments — Fresher/Interns and Lateral Hires (3–4+ years experience) — with tailored journeys for each.

- Freshers land in a Knowledge Factory — a simple flow where they submit their resume and contact details.
- Lateral candidates browse curated job postings; AI matches their resume to the best-fit roles before they apply.
- An AI-powered ATS engine evaluates every lateral application against the job description, auto-sends email notifications, and surfaces shortlisted candidates on the Recruiter Dashboard.
- Recruiters at Coastel Seven get a ranked, scored view of all qualified applicants.

2. Problem Statement & Goals

2.1 Problem Statement

Coastel Seven lacks a centralised digital hiring channel. Resumes arrive via email without automated screening, making it difficult to rank candidates or scale hiring.

2.2 Goals

1. Provide a self-serve portal for freshers to register interest with zero friction.
2. Enable lateral candidates to discover relevant jobs with AI-assisted matching.
3. Automate ATS screening and candidate communication to reduce recruiter workload by 60%.
4. Give recruiters a real-time, ranked pipeline of qualified candidates per job.

3. Scope

3.1 In Scope

- Public-facing Career Portal with two candidate pathways (Fresher & Lateral)
- Knowledge Factory flow: resume upload + contact capture
- Lateral job board: dynamic listings with AI resume-to-job matching
- AI ATS Engine: resume vs. JD evaluation, relevance scoring, auto-email
- Recruiter Dashboard: filtered, scored candidate pipeline per job
- Automated email notification system
- Admin panel for managing job postings (CRUD)

3.2 Out of Scope (v1.0)

- Interview scheduling & calendar integration
- Offer letter generation
- Candidate login / profile persistence across sessions
- Mobile native app (portal will be mobile-responsive)

4. Feature Requirements

4.1 Landing Page

Entry point with two clear paths: Fresher/Intern and Lateral Hire. No login required.

4.2 Knowledge Factory (Fresher Flow)

- Full Name, Email, Phone, College, Graduation Year, Resume Upload, Optional Note
- On submit: confirmation message + acknowledgement email to candidate
- Resume stored in secure cloud storage (S3)

4.3 Job Board & AI Matching (Lateral Flow)

- Step 1: Upload resume — AI parses skills, experience, tech stack
- Step 2: AI matches resume against all active jobs, returns ranked list with Match %
- Step 3: Job detail view — title, dept, experience, location, match score, Apply CTA
- Step 4: Apply to Individual Job OR Apply to All Matched Jobs (>= 60% threshold)

4.4 AI ATS Engine

- Fires automatically on every lateral application
- Evaluates: skill alignment, experience match, keyword density, role-title fit
- Generates Relevance Score (0-100)
- Score >= 60: Under Review email | Score < 60: Rejected email
- Emails dispatched within 2 minutes of submission

4.5 Recruiter Dashboard

- Login-protected; JWT + RBAC
- Job pipeline view sorted by Relevance Score descending
- Filter by job, score range, date | Export CSV/Excel | Inline PDF resume viewer

5. User Stories

#	User Story	Acceptance Criteria	Priority
US-01	As a fresher, I want to submit my resume in under 2 minutes.	Form succeeds; confirmation email received; entry visible in admin.	**High**
US-02	As a lateral candidate, I want to see jobs that match my profile.	Matched jobs with % score shown within 10 seconds of upload.	**High**
US-03	As a lateral candidate, I want to apply to all matched jobs with one click.	Apply to All triggers applications >= threshold; modal shown.	**High**
US-04	As a lateral candidate, I want an email after applying so I know my status.	Email with score and status received within 2 minutes.	**High**

US-05	As a recruiter, I want shortlisted candidates ranked by score per job.	Dashboard shows ranked list; filter by job works.	**High**
US-06	As a recruiter, I want to export a candidate list to Excel.	Export downloads a valid .xlsx with correct data.	**Medium**
US-07	As an admin, I want to create and edit job postings.	Job published within 1 minute; appears on lateral board.	**High**
US-08	As an admin, I want to configure the ATS score threshold.	Threshold change takes effect on next application.	**Medium**

6. API Reference

All endpoints are prefixed with /api. All responses follow a normalized format: { status, data, message }. Authentication via JWT Bearer token for protected routes.

6.1 Common — Resume Upload

Shared by both fresher and lateral flows. Must be called first before any other action.

POST `/api/resume/upload`

Accepts a multipart form upload. Parses the resume using NLP and returns a resumeId plus structured data.

Request Body

```
Content-Type: multipart/form-data

file: resume.pdf   // PDF or DOCX, max 5MB
```

Response (200 OK)

```
{
  "resumeId": "res_123",
  "parsedData": {
    "skills": ["Python", "Spring Boot"],
    "experience": 3,
    "currentRole": "Software Engineer",
    "education": "B.Tech Computer Science"
  }
}
```

6.2 Fresher / Intern Flow

POST `/api/fresher/register`

Registers a fresher candidate into the Knowledge Factory. Links their parsed resume to their profile.

Request Body

```
{
  "name": "Lokesh",
  "email": "lokesh@email.com",
  "phone": "9999999999",
  "college": "JNTU Hyderabad",
  "graduationYear": 2025,
  "resumeId": "res_123"
}
```

Response (200 OK)

```
{
  "status": "success",
  "message": "Successfully registered in Knowledge Factory",
  "candidateId": "cand_fresher_001"
}
```

GET `/api/fresher/{id}`

Future use — retrieves a fresher candidate profile by ID. Currently for admin use only.

Response (200 OK)

```
{
  "id": "cand_fresher_001",
  "name": "Lokesh",
  "email": "lokesh@email.com",
  "resumeId": "res_123",
  "registeredAt": "2026-04-06T10:00:00Z"
}
```

6.3 Lateral Candidate Flow

POST `/api/jobs/match`

CORE FEATURE. Takes a parsed resumeId and returns a ranked list of matched jobs with scores, strengths, and missing skills.

Request Body

```
{
  "resumeId": "res_123"
}
```

Response (200 OK)

```
[
  {
    "jobId": "job_1",
    "title": "Backend Developer",
    "department": "Engineering",
    "location": "Hyderabad",
    "matchScore": 82,
    "missingSkills": ["Docker"],
    "strengths": ["Java", "Spring Boot"]
  },
  {
```

```
    "jobId": "job_2",
    "title": "Full Stack Developer",
    "matchScore": 71,
    "missingSkills": ["React"],
    "strengths": ["Python", "REST APIs"]
  }
]
```

POST `/api/applications/apply`

Applies a lateral candidate to a single job posting. Triggers the ATS evaluation pipeline in the background.

Request Body

```
{
  "candidateId": "cand_101",
  "jobId": "job_1",
  "resumeId": "res_123"
}
```

Response (200 OK)

```
{
  "status": "success",
  "applicationId": "app_456",
  "message": "Application submitted. You will receive an email shortly."
}
```

POST `/api/applications/apply-bulk`

Applies a lateral candidate to multiple jobs at once. Ideal for the Apply to All Matched Jobs feature.

Request Body

```
{
  "candidateId": "cand_101",
  "jobIds": ["job_1", "job_2", "job_3"],
  "resumeId": "res_123"
}
```

Response (200 OK)

```
{
  "status": "success",
  "appliedTo": ["job_1", "job_2", "job_3"],
  "failedJobs": [],
  "message": "Applied to 3 jobs. Emails will be sent shortly."
}
```

6.4 ATS + AI Evaluation

This endpoint is called INTERNALLY by the backend after any application. Do NOT expose to frontend. It can be called directly for debugging and testing purposes.

POST `/api/ats/evaluate`

Internal endpoint. Evaluates a resume against a job description and returns a structured score with reasoning.

Request Body

```
{
  "resumeId": "res_123",
  "jobId": "job_1"
}
```

Response (200 OK)

```
{
  "score": 78,
  "status": "UNDER_REVIEW",
  "missingSkills": ["AWS", "Kubernetes"],
  "matchedSkills": ["Java", "Spring Boot", "REST APIs"],
  "summary": "Strong backend experience but lacks cloud exposure.",
  "recommendation": "Shortlist for technical screening"
}
```

6.5 Recruiter Dashboard

All recruiter endpoints require Authorization: Bearer <jwt_token> header.

GET `/api/recruiter/jobs`

Returns all job postings with applicant count and shortlist summary.

Response (200 OK)

```
[
  {
    "jobId": "job_1",
    "title": "Backend Developer",
    "totalApplicants": 42,
    "shortlisted": 8,
    "status": "ACTIVE"
  }
]
```

GET `/api/recruiter/jobs/{jobId}/candidates`

Returns all candidates who applied to a specific job, sorted by ATS relevance score descending.

Response (200 OK)

```
[
  {
    "candidateId": "cand_101",
    "name": "Lokesh",
    "email": "lokesh@email.com",
    "score": 82,
    "status": "SHORTLISTED",
    "appliedAt": "2026-04-05T09:30:00Z",
    "resumeUrl": "https://s3.../res_123.pdf"
  }
]
```

POST `/api/recruiter/filter`

Filters candidates across a job posting by minimum score, skills, and status.

Request Body

```
{
  "jobId": "job_1",
  "minScore": 70,
  "skills": ["Python"],
  "status": "SHORTLISTED"
}
```

Response (200 OK)

```
{
  "status": "success",
  "count": 12,
  "candidates": [ ...filtered list... ]
}
```

6.6 Job Management (Admin)

POST `/api/jobs`

Creates a new job posting. Requires admin/recruiter JWT token.

Request Body

```
{
  "title": "Backend Developer",
  "department": "Engineering",
  "location": "Hyderabad",
  "experienceMin": 3,
  "experienceMax": 5,
  "description": "Full JD text here...",
  "requiredSkills": ["Java", "Spring Boot", "SQL"],
  "status": "ACTIVE"
}
```

Response (200 OK)

```
{
  "status": "success",
  "jobId": "job_5",
  "message": "Job posted successfully"
}
```

GET `/api/jobs`

Public endpoint. Returns all active job postings for the lateral candidate job board.

Response (200 OK)

```
[
  {
    "jobId": "job_1",
    "title": "Backend Developer",
```

```
    "location": "Hyderabad",
    "experienceRequired": "3-5 years",
    "skills": ["Java", "Spring Boot"]
  }
]
```

GET `/api/jobs/{jobId}`

Public endpoint. Returns full details of a single job posting.

Response (200 OK)

```
{
  "jobId": "job_1",
  "title": "Backend Developer",
  "description": "Full JD...",
  "requiredSkills": ["Java", "Spring Boot"],
  "location": "Hyderabad",
  "status": "ACTIVE"
}
```

6.7 Email Notification System (Internal)

This endpoint is triggered internally by the backend after ATS evaluation. The frontend never calls this directly.

POST `/api/notifications/email`

Sends a transactional email to a candidate. Triggered automatically on: application received, shortlisted, or rejected.

Request Body

```
{
  "to": "lokesh@email.com",
  "type": "UNDER_REVIEW" | "REJECTED" | "SHORTLISTED",
  "candidateName": "Lokesh",
  "jobTitle": "Backend Developer",
  "score": 78,
  "summary": "Strong backend profile, missing cloud skills"
}
```

Response (200 OK)

```
{
  "status": "success",
  "messageId": "msg_789",
  "deliveredAt": "2026-04-06T10:02:15Z"
}
```

7. Backend Application Flow

This section describes the exact sequence of operations that happen when a lateral candidate applies for a job. Every junior developer must understand this flow completely before writing code.

7.1 Full Lateral Application Flow — Step by Step

1

Candidate Uploads Resume

`POST /api/resume/upload`

File saved to S3. Text extracted via pdf-parse / docx2txt. AI parses skills, experience, education. Returns resumeId.

2

AI Job Matching

`POST /api/jobs/match`

Backend fetches all active jobs from DB. AI compares parsed resume against each JD. Returns ranked list with matchScore, strengths, missingSkills.

3

Candidate Applies

`POST /api/applications/apply OR /apply-bulk`

Application record(s) saved to DB with status PENDING. A job is added to the async processing queue (BullMQ / SQS).

4

ATS Evaluation (Background Job)

`Internal: POST /api/ats/evaluate`

Queue worker picks up the job. Calls AI API with resume text + job JD. Gets back score, status, summary, missingSkills.

5

Score Stored & Email Triggered

`Internal: POST /api/notifications/email`

ATS score saved to applications table. If score >= threshold: status = UNDER_REVIEW. Else: status = REJECTED. Email dispatched via SendGrid.

6

Recruiter Views Ranked Candidates

`GET /api/recruiter/jobs/{jobId}/candidates`

Recruiter dashboard fetches candidates sorted by score DESC. Only candidates with status != REJECTED appear in top list.

7.2 Fresher Registration Flow

1

Fresher Uploads Resume

`POST /api/resume/upload`

Same as lateral. Returns resumeId.

2	**Fresher Submits Details** `POST /api/fresher/register` *Candidate record created in DB. resumeId linked. Acknowledgement email sent. No ATS evaluation needed.*

8. Database Schema

PostgreSQL is the primary database. All tables use UUID primary keys. Timestamps in UTC. The schema below covers the core tables for v1.0.

candidates

Column	Type	Description
id	`UUID (PK)`	Primary key, auto-generated
name	`VARCHAR(255)`	Full name of the candidate
email	`VARCHAR(255)`	Email address (unique)
phone	`VARCHAR(20)`	Phone number (optional)
type	`ENUM('FRESHER','LATERAL')`	Candidate category
college	`VARCHAR(255)`	College/university (fresher only)
grad_year	`INTEGER`	Graduation year (fresher only)
created_at	`TIMESTAMP`	Registration timestamp (UTC)

resumes

Column	Type	Description
id	`UUID (PK)`	Primary key — the resumeId returned by /upload
candidate_id	`UUID (FK → candidates)`	Owner of this resume
file_url	`TEXT`	S3 URL for the resume file
parsed_skills	`JSONB`	Array of extracted skills
experience_years	`INTEGER`	Years of experience extracted
current_role	`VARCHAR(255)`	Current job title if found
raw_text	`TEXT`	Full extracted text for AI prompts
created_at	`TIMESTAMP`	Upload timestamp

jobs

Column	Type	Description
id	UUID (PK)	Primary key — the jobId
title	VARCHAR(255)	Job title
department	VARCHAR(100)	Department name
location	VARCHAR(100)	Office location
description	TEXT	Full job description (used by ATS)
required_skills	JSONB	Array of required skill strings
experience_min	INTEGER	Minimum years of experience
experience_max	INTEGER	Maximum years of experience
status	ENUM('ACTIVE', 'CLOSED')	Whether the job is open
created_at	TIMESTAMP	Job creation timestamp
created_by	UUID (FK → recruiters)	Who posted this job

applications

Column	Type	Description
id	UUID (PK)	Application ID
candidate_id	UUID (FK → candidates)	Who applied
job_id	UUID (FK → jobs)	Which job
resume_id	UUID (FK → resumes)	Which resume was used
ats_score	INTEGER	AI-generated relevance score (0-100)
status	ENUM('PENDING','UNDER_REVIEW','SHORTLISTED','REJECTED')	Current status
ats_summary	TEXT	AI-generated evaluation summary
missing_skills	JSONB	Skills the candidate lacks
email_sent_at	TIMESTAMP	When notification email was dispatched
applied_at	TIMESTAMP	Application submission time

match_scores

Column	Type	Description
id	UUID (PK)	Match record ID
resume_id	UUID (FK → resumes)	Resume that was matched

job_id	UUID (FK → jobs)	Job that was matched against
match_score	INTEGER	Match percentage (0-100)
strengths	JSONB	Skills that matched well
missing_skills	JSONB	Skills not found in resume
computed_at	TIMESTAMP	When the match was computed

9. Developer Guide — Do's & Don'ts

This section is specifically written for junior developers joining this project. Read this before writing a single line of code. These rules exist because of real mistakes that have been made on similar projects.

9.1 API Design Rules

DO (Best Practices)	DON'T (Common Mistakes)
✔ Always return { status, data, message } in every response	✘ Don't return raw AI output directly to the frontend
✔ Use HTTP status codes correctly (200, 201, 400, 401, 404, 500)	✘ Don't mix fresher + lateral business logic in the same function
✔ Validate all request body fields before processing	✘ Don't hardcode ATS threshold scores in code — put it in config/DB
✔ Return meaningful error messages with field-level details	✘ Don't call AI API synchronously inside a request — use a queue
✔ Keep fresher and lateral logic in separate route files	✘ Don't expose /api/ats/evaluate or /api/notifications/email publicly
✔ Use async/await with proper try-catch in every controller	✘ Don't return stack traces or internal errors to the client
✔ Log all AI API calls with request/response for debugging	✘ Don't skip input validation — malformed resumes will crash parsers

9.2 AI & ATS Engine Rules

DO (Best Practices)	DON'T (Common Mistakes)
✔ Keep AI as a separate service / module — not mixed in controllers	✘ Don't trust AI output without sanitizing / validating the schema

✔ Always normalize AI scores to 0-100 range before storing	✘ Don't call AI API directly from the application controller
✔ Have a fallback if AI API times out (retry with exponential backoff)	✘ Don't hardcode scoring logic — keep it prompt-driven and configurable
✔ Store raw AI response in DB for audit and debugging purposes	✘ Don't skip the queue — ATS evaluation must be async, not blocking
✔ Test prompts with at least 10 real resumes before go-live	✘ Don't show raw AI summaries to candidates without formatting
✔ Allow the ATS threshold to be configured per job posting	

9.3 Database Rules

DO (Best Practices)	DON'T (Common Mistakes)
✔ Use UUIDs for all primary keys — never sequential integers	✘ Don't store resume raw text in the candidates table — use resumes table
✔ Always index foreign keys and frequently filtered columns	✘ Don't delete application records — soft delete with a status flag
✔ Use JSONB for flexible arrays (skills, missingSkills, strengths)	✘ Don't run raw SQL strings — always use parameterized queries / ORM
✔ Store all timestamps in UTC	✘ Don't store passwords or tokens in plain text
✔ Write DB migrations for every schema change — never edit live DB	✘ Don't skip foreign key constraints — they protect data integrity
✔ Use transactions when writing to multiple tables at once	

9.4 Email System Rules

DO (Best Practices)	DON'T (Common Mistakes)
✔ Always send emails from a queue — never inline in the request cycle	✘ Don't send emails synchronously — it will slow down the API response
✔ Use HTML email templates with Coastel Seven branding	✘ Don't hardcode email templates in code — use a template engine
✔ Include the job title and score band in every candidate email	✘ Don't expose the exact ATS score in rejection emails (use bands: Low/Medium/High)
✔ Log email delivery status and message ID for tracking	✘ Don't skip email validation before dispatch
✔ Test emails in staging before enabling on production	

9.5 Security Rules

DO (Best Practices)	DON'T (Common Mistakes)
✔ Protect all recruiter and admin endpoints with JWT middleware	✘ Don't commit .env files or API keys to Git — ever
✔ Use pre-signed S3 URLs for resume file access — never public buckets	✘ Don't trust file extension alone for upload validation — check MIME type
✔ Rate limit the /resume/upload and /fresher/register endpoints	✘ Don't allow unauthenticated access to recruiter or admin endpoints
✔ Validate file type AND file content on upload (not just extension)	✘ Don't store AI API keys in frontend code
✔ Use environment variables for all API keys and secrets	✘ Don't use wildcard (*) for CORS in production
✔ Add CORS whitelist — only allow your frontend domain	

9.6 Response Format Standard

ALL API responses must follow this exact format. No exceptions.

```
// SUCCESS RESPONSE
{
  "status": "success",
  "data": { ...actual data here... },
  "message": "Human-readable success message"
}

// ERROR RESPONSE
{
  "status": "error",
  "data": null,
  "message": "Human-readable error message",
  "errors": [
    { "field": "email", "message": "Invalid email format" }
  ]
}
```

9.7 Folder Structure Recommendation

```
src/
  routes/
    fresher.routes.js        // POST /api/fresher/*
    lateral.routes.js        // POST /api/jobs/*, /api/applications/*
    recruiter.routes.js      // GET /api/recruiter/* (protected)
    admin.routes.js          // POST /api/jobs (protected)
    resume.routes.js         // POST /api/resume/upload
```

```
  controllers/
    fresher.controller.js
    lateral.controller.js
    recruiter.controller.js
    resume.controller.js
  services/
    ats.service.js           // AI scoring logic — isolated here
    matching.service.js      // Job matching logic
    email.service.js         // Email dispatch via SendGrid
    resume.service.js        // Parse + extract text
  queues/
    ats.queue.js             // BullMQ worker for async ATS jobs
    email.queue.js           // BullMQ worker for email dispatch
  models/                    // DB models / ORM schema
  middleware/
    auth.middleware.js       // JWT validation
    upload.middleware.js     // Multer / S3 config
  config/
    db.js                   // DB connection
    ai.js                   // AI API client config
  utils/
    response.util.js        // Normalized { status, data, message }
    validate.util.js        // Input validation helpers
```

10. Technical Architecture

10.1 Recommended Stack

Frontend	React.js (Next.js) + Tailwind CSS
Backend / API	Node.js (Express) or Python (FastAPI)
Database	PostgreSQL + Redis (cache/sessions)
File Storage	AWS S3 — resume files with pre-signed URLs
AI Engine	OpenAI GPT-4 API or Anthropic Claude API
Email Service	SendGrid or AWS SES
Queue System	BullMQ (Node) or Celery (Python)
Auth	JWT + RBAC for recruiter/admin access
Hosting	AWS EC2/ECS + RDS / Vercel (frontend)
CI/CD	GitHub Actions → staging → production

11. Action Plan & Project Roadmap

12-week delivery across 5 phases. Each phase builds on the previous.

Phase	Task	Owner	Duration	Priority
Phase 1	Project setup — repo, infra, CI/CD pipeline	Tech Lead	Week 1	High
Phase 1	UI/UX design — wireframes & design system	Designer	Week 1-2	High
Phase 1	Database schema design + migrations	Backend Dev	Week 1	High
Phase 2	Landing page + candidate type selection	Frontend Dev	Week 2	High
Phase 2	Resume upload service (S3 + parser)	Backend Dev	Week 2-3	High
Phase 2	Knowledge Factory form + fresher API	Full Stack Dev	Week 3	High
Phase 3	AI Job Matching — prompt engineering	AI Engineer	Week 4-5	High
Phase 3	Lateral Job Board UI — cards, match scores	Frontend Dev	Week 4-5	High
Phase 3	Apply Individual + Apply Bulk APIs	Backend Dev	Week 5	High
Phase 4	ATS Engine — async queue + scoring	AI Engineer	Week 5-6	High
Phase 4	Email notification system (SendGrid)	Backend Dev	Week 6	High
Phase 4	Recruiter Dashboard — pipeline + auth	Full Stack Dev	Week 6-7	High
Phase 4	Candidate filter + export CSV/Excel	Frontend Dev	Week 7	Medium
Phase 5	Admin panel — job CRUD + config	Full Stack Dev	Week 7-8	High
Phase 5	QA — functional, integration, UAT	QA Engineer	Week 9-10	High
Phase 5	Security audit + penetration test	DevOps	Week 10-11	High
Phase 5	Production deployment + go-live	DevOps	Week 12	High

12. Risks & Mitigations

Risk	Severity	Mitigation
AI scoring inaccuracy or bias	High	Thorough prompt testing with real resumes; human override always available
Email delivery delays or spam filtering	Medium	Dedicated sending domain; SPF/DKIM; monitor via SendGrid dashboard

Resume parsing fails on complex formats	**Medium**	Multi-library fallback (pdfplumber + OCR via Tesseract)
Async queue failures causing missed emails	**High**	Dead-letter queue; retry with exponential backoff; alerting on failures
Data privacy breach (resume data)	**High**	Encryption at rest + in transit; pre-signed URLs; penetration test pre-launch
Scope creep mid-project	**Medium**	Strict change control; all additions require written approval

13. Approvals & Sign-Off

Role	Name	Signature	Date
Project Sponsor (Manager)			
HR Lead (Coastel Seven)			
Tech Lead			
Product Manager			

v1.0	April 2026 — Initial PRD
v2.0	April 2026 — Added API Reference, Backend Flow, DB Schema, Developer Guide

V3.0 (Important Changes on Scoring):

🔧 ADDITIONS TO PRD — ATS SCORING & PARSING IMPROVEMENTS

---

1. Resume Parsing Module (Updated Design)

Objective

Ensure reliable extraction of structured data from resumes while minimizing parsing failures and inconsistencies.

Updated Approach

- Use a dedicated parsing module (Python microservice or Node-based library)

- Extract the following fields:
    - Skills (normalized)
    - Experience (years)
    - Current Role
    - Education
    - Raw Text (mandatory for fallback processing)

Key Improvements

- Add **post-processing layer**:
    - Normalize skill names ("JS" → "JavaScript")
    - Remove duplicates
    - Map synonyms
- Add **fallback parsing strategy**:
    - Primary: pdf-parse / docx parser
    - Secondary: OCR (Tesseract) for malformed resumes
- Store:
    - Parsed structured data
    - Raw resume text (for scoring + keyword matching)

Architecture

Resume Upload → Parser → Post-Processing → DB Storage

---

2. ATS Scoring System (MAJOR CHANGE)

❌ Removed

- AI-based direct scoring (due to inconsistency, hallucination, and lack of determinism)

✅ New Approach: Rule-Based Scoring Engine

The ATS scoring system will be fully deterministic and explainable.

---

3. Scoring Components

Each application is evaluated using weighted components:

Component	Description	Weight

Skill Match	Overlap between resume & required skills	50%
Experience Match	Alignment with required experience range	30%
Keyword Match	Match with job description keywords	20%

---

4. Scoring Logic

Skill Match

score = (matched_skills / required_skills) * weight

Experience Match

- Full score if within range
- Partial score if below minimum

Keyword Match

- Based on keyword frequency in resume raw text

Final Score

Final Score = Skill Score + Experience Score + Keyword Score

- Output normalized to range: 0–100
- Same input must always produce the same output

---

5. Score Breakdown (NEW FEATURE)

To improve transparency and recruiter trust:

Response will include:

{
"score": 78,
"breakdown": {
"skills": 40,
"experience": 25,
"keywords": 13

}
}

---

6. Configurable Scoring Weights (NEW FEATURE)

Weights should NOT be hardcoded.

Store in DB or config:

{
"skillWeight": 50,
"experienceWeight": 30,
"keywordWeight": 20
}

This allows recruiters/admins to adjust scoring logic dynamically.

---

7. AI Role (Revised)

AI will NOT be used for scoring.

AI will ONLY be used for:

- Generating candidate summaries
- Suggesting missing skills
- Enhancing recruiter insights

Example:
{
"summary": "Strong backend experience but lacks cloud exposure"
}

---

8. Updated ATS Evaluation Flow

Previous:
AI → Score → Store ❌

Updated:

Resume → Parser → Rule Engine → Score
↓
(Optional)
↓
AI Summary

---

9. Benefits of New Approach

- Deterministic scoring (no randomness)
- Explainable results (breakdown available)
- Faster execution (no AI dependency)
- Easier debugging and testing
- Higher recruiter trust

---

10. Future Enhancements (Optional)

- Add embedding-based similarity for semantic matching
- Add confidence score
- Add recruiter feedback loop to improve scoring

---

11. Developer Notes

- Do NOT rely on AI output for numeric scoring
- Always validate parsed data before scoring
- Keep scoring logic modular (separate service)
- Ensure consistent normalization across all resumes

---

12. Impact on Existing APIs

/api/ats/evaluate (Updated Response)

{
"score": 78,
"status": "UNDER_REVIEW",

"breakdown": {
"skills": 40,
"experience": 25,
"keywords": 13
},
"missingSkills": ["AWS"],
"summary": "Strong backend profile, missing cloud skills"
}

This update ensures the ATS system is reliable, explainable, and production-ready while keeping AI usage controlled and safe.